

Urban Sound Classification with Classical Machine Learning and Deep Learning

A Alkaff Ahamed, Dev Akhbar, Wong Kee Fong

1. Introduction and Background

Environmental sound classification is a standard benchmark for comparing classical machine learning (ML) and deep learning (DL). In this project, we built an end-to-end system to classify 10 types of urban sounds (e.g. air conditioners, sirens, drilling, street music) from short audio clips.

Our dataset contains 2 597 clips, split into 1 760 training and 837 test examples across 10 classes. The distribution is moderately imbalanced (e.g. *jackhammer* is much more common than *gun_shot*), so we evaluate models using overall accuracy and macro-averaged precision, recall and F1-score. This prevents majority classes from dominating the metrics.

Prior work typically uses CNNs on log-mel spectrograms, sometimes with recurrent or attention layers, while older pipelines rely on MFCC features with classical classifiers. We re-implement both approaches on the same preprocessed data to address:

Research question. On a medium-sized, moderately imbalanced urban-sound dataset, does a modern tree-based MFCC pipeline still compete with compact CNN/CRNN spectrogram models?

We compare MFCC-based classical models (Logistic Regression, SVM, HistGradientBoosting, XGBoost) against CNN and CRNN architectures trained on log-mel spectrograms.

2. Methodology

2.1 Data Preprocessing Pipeline

We design a single pipeline that feeds both MFCC-based and spectrogram-based models, and apply outlier detection once, then propagate removals consistently across all views.

MFCC Branch

- Raw audio → **loudness normalisation**

- Compute **40 MFCCs** per frame plus **1st and 2nd derivatives** (Δ , $\Delta\Delta$), giving **120 MFCC-related time series**
- For each of the 120 series, **compute 10 statistics** (mean, median, quartiles, IQR, standard deviation, skewness, kurtosis, min, max), producing a **1 200-dimensional static vector** (MFCC-1200) per clip, **saved to CSV**
- Standardise **MFCC-1200** using **RobustScaler** and apply **PCA to 50 components**, yielding MFCC-PCA-50 CSVs used by linear models

Spectrogram and Waveform Branches

- Raw audio → **loudness normalisation** → **repeat-fill** shorter clips and **crop** longer clips to 4 s so all clips have **equal duration**
- From these 4 s segments, compute a **mel spectrogram** (128 mel bins), apply **log scaling** and reshape/interpolate to a fixed 186×128 tensor (time $\times$ frequency)
- **Save** the tensors as **grayscale PNGs** for CNN/CRNN models
- In parallel, create a **24 kHz downsampled waveform** version of the same 4 s segment for potential waveform-based models

Outlier Detection and Synchronised Cleaning

- **Load MFCC-PCA-50** training features and fit **Isolation Forest (IF)** and **Local Outlier Factor (LOF)**
- Define the final outlier set as the intersection **IF $\cap$ LOF**. This flags 39 of 1 760 training clips (~2.2 %), leaving 1 721 clips
- Use **t-SNE** and **UMAP (cosine distance)** on MFCC-PCA-50 to **visually confirm** that consensus outliers lie near the fringes of class clusters
- **Remove** the consensus outliers from **all representations**: downsampled waveforms, MFCC-1200 CSVs, MFCC-PCA-50 CSVs and spectrogram PNGs. The test set (837 clips) is never altered

2.2 Modeling

Classical ML (MFCC Features)

- **Logistic Regression** and **SVM with RBF kernel** are trained on **MFCC-PCA-50**, with class weights and coarse grid search for regularisation strength and kernel parameters.

- **HistGradientBoosting** and **XGBoost** are trained on the full **MFCC-1200** features, using class weighting, early stopping and hyperparameter tuning (number of trees, depth, learning rate, subsampling) with macro-F1 as the objective.
- After tuning, all models are retrained on the cleaned 1 721-clip training set and evaluated once on the 837-clip test set.

Deep Learning (Log-Mel Spectrograms)

- **Inputs** are 186×128 log-mel spectrograms with a single channel.
- We create an **80 % / 20 % stratified train/validation** split on the cleaned training set. the test set remains held out.
- We train:
 - A **3-block CNN** (Conv-BN-ReLU-MaxPool $\times 3$, GlobalAveragePooling, dropout, dense softmax).
 - A **4-block CNN** that adds one extra convolutional block.
 - A **CRNN** with convolutional front-end, GRU layer and dense softmax head.
- All networks are implemented in TensorFlow/Keras, trained with Adam, class-weighted cross-entropy and early stopping on validation performance.

3. Results and Discussion

On the 837-clip test set, the best classical model (XGBoost on MFCC-1200) slightly outperforms the best deep model:

- **XGBoost (MFCC-1200)** – accuracy 70.1 %, macro-F1 71.1 %
- **CNN (4 blocks, log-mel spectrogram)** – accuracy 67.7 %, macro-F1 66.7 %

Other baselines obtain:

- **Logistic Regression** (MFCC-PCA-50) – 60.7 % accuracy, 61.2 % macro-F1
- **SVM with RBF kernel** (MFCC-PCA-50) – 60.6 % accuracy, 61.6 % macro-F1
- **HistGradientBoosting** (MFCC-1200) – 68.5 % accuracy, 69.0 % macro-F1
- **CNN (3 blocks)** – 67.0 % accuracy, 66.3 % macro-F1
- **CRNN** – 62.1 % accuracy, 64.5 % macro-F1

From these results we observe:

1. **MFCC + tree-based models are very strong.** XGBoost on MFCC-1200 achieves the highest macro-F1 despite using only summary statistics of MFCC trajectories
2. **CNNs close most of the gap** but do not dominate. The 4-block CNN performs competitively, but still slightly below XGBoost, suggesting that additional data or augmentation would be needed for CNNs to clearly win
3. **CRNN does not pay off** at this data scale. The recurrent layer adds capacity but does not translate into higher macro-F1 with $\sim 1.7k$ training examples
4. **Minority classes remain challenging.** Confusion matrices show that rarer sounds (e.g. *gun_shot*, *car_horn*) are still harder to recognise, which is reflected in the macro-averaged metrics

4. Conclusion

We built a unified pipeline for urban sound classification that supports both **MFCC-based classical ML** and **log-mel spectrogram-based CNN/CRNN models**, with a **shared preprocessing and outlier-removal** strategy. On our 10-class, $\sim 2.6k$ -clip dataset, a **well-tuned MFCC + XGBoost model** slightly outperforms compact CNN and CRNN architectures in macro-F1.

This suggests that, when data is limited and carefully preprocessed, feature-engineered MFCCs plus strong tree-based learners remain highly competitive even against modern deep models. CNNs still offer flexibility and may outperform classical models once they can exploit more data and augmentation.

5. Future Enhancements

To further improve performance and robustness, we propose:

1. **Data augmentation** – waveform-level (time shifting, noise, pitch/tempo changes) and spectrogram-level (mixup, time/frequency masking).
2. **Richer temporal features for classical ML** – segment MFCC time series and summarise each segment to capture coarse temporal “shape”.
3. **Transfer learning and stronger DL backbones** – fine-tune audio-pretrained CNNs or TCNs and explore multimodal ensembles that combine MFCC-based XGBoost with CNN/CRNN predictions.

Appendix A – Dataset and preprocessing details

A.1 Class distribution (train vs test)

The per-class distribution used in our experiments is:

Class	Train	Test
air_conditioner	200	100
car_horn	78	33
children_playing	200	100
dog_bark	200	100
drilling	200	100
engine_idling	196	93
gun_shot	70	32
jackhammer	239	96
siren	177	83
street_music	200	100

Totals:

- **Training:** 1 760 clips before outlier removal; **1 721** after removing 39 consensus outliers
- **Test:** 837 clips (unchanged by outlier removal)

A.2 Outlier detection summary

- We run **Isolation Forest** and **Local Outlier Factor** on the MFCC-PCA-50 training features
- The consensus outlier set is defined as **IF** $\cap$ **LOF**, giving **39 out of 1 760** clips (~2.2 %) flagged as outliers
- These outliers are removed from **all** representations (downsampled audio, MFCC-1200, MFCC-PCA-50, and spectrogram PNGs), ensuring consistent data across the pipeline

Appendix B – Models and evaluation metrics

B.1 Classical ML Macro Metrics (Test Set)

Model	Features	Acc	Macro Prec	Macro Rec	Macro F1
Logistic Regression	MFCC-PCA-50	60.7%	63.4%	60.9%	61.2%
SVM (RBF)	MFCC-PCA-50	60.6%	65.1%	61.2%	61.6%
HistGradientBoost	MFCC-1200	68.5%	73.5%	67.8%	69.0%
XGBoost	MFCC-1200	70.1%	74.4%	69.9%	71.1%

- Metrics are computed on the **837-clip test set**
- Macro precision, recall and F1 are averaged uniformly across the 10 classes

B.2 Deep Learning Macro Metrics (Test Set)

Model	Input	Acc	Macro Prec	Macro Rec	Macro F1
CNN (3 Blocks)	186×128 log-mel spectrogram	67.0%	68.3%	67.4%	66.3%
CNN (4 Blocks)	186×128 log-mel spectrogram	67.7%	67.3%	69.1%	66.7%
CRNN	186×128 log-mel spectrogram	62.1%	65.7%	63.9%	64.5%

- CNN and CRNN models are selected using validation macro-F1 and then evaluated once on the test set
- Inputs are log-mel spectrograms of size 186×128 with one channel

B.3 CNN and CRNN Architecture Details (Summary)

- **CNN (3 blocks)**
 - Input: (186, 128, 1)
 - Block 1: Conv2D(32, 3×3) → BatchNorm → ReLU → MaxPool2D
 - Block 2: Conv2D(64, 3×3) → BatchNorm → ReLU → MaxPool2D
 - Block 3: Conv2D(128, 3×3) → BatchNorm → ReLU → MaxPool2D
 - Head: GlobalAveragePooling2D → Dropout → Dense(10, softmax)
- **CNN (4 blocks)**
 - Extends the 3-block CNN with an additional Conv2D–BatchNorm–ReLU–MaxPool2D block at higher filter counts
- **CRNN**
 - Two Conv2D–BatchNorm–ReLU–MaxPool2D blocks to compress the frequency axis while retaining temporal resolution
 - Third Conv2D–BatchNorm–ReLU–MaxPool2D blocks to compress the frequency axis and time axis to compress the timesteps slightly
 - Projection and reshape to a sequence and feed into a GRU layer to model temporal context
 - Dense(10, softmax) output layer
- All models are implemented in TensorFlow/Keras and trained with class-weighted cross-entropy and early stopping on validation macro-F1

Appendix C – Others

Computational Environment

Hardware: GPU: NVIDIA RTX 3050 (4GB VRAM), CPU: Intel i7-11800H (8 cores), RAM: 32GB DDR4

Software: Python 3.9.21, Keras 2.10 (TensorFlow 2.10 backend) with CUDA 11.2 and cuDNN 8.1, librosa 0.11.0, scikit-learn 1.6.1, umap-learn 0.5.9, xgboost 2.1.3

Acknowledgements

AI Tools: Report prepared with AI assistance (Gemini Pro, Claude) for structure optimization, technical writing, and proofreading. Notebooks prepared with a custom template. AI assistance (ChatGPT) was used for assistance in interpreting some results and fine-tuning some insights. All technical content, ideas, experimental design, analysis, and conclusions are original work of the authors.

Additional: We thank Professor Prabhu and the MSI5001 teaching team for guidance throughout this project.